Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology**	
Subject: DSIP	**Aim: Image processing_3**	
Experiment:-9	**Date:22-08-2026**	**Enrollment No:92400133063**

Aim:- **write a python program to simulate smoothening and sharpening operation on image using spatial filter**

Code:-

```
import cv2
import numpy as np

# Load the image
image = cv2.imread('free-nature-images.jpg')

# Define the Gaussian kernel for smoothing
kernel_size = (5, 5)
sigma = 1.5
gaussian_kernel = cv2.getGaussianKernel(kernel_size[0], sigma)
gaussian_kernel = np.outer(gaussian_kernel, gaussian_kernel)

# Apply Gaussian smoothing
smoothed_image = cv2.filter2D(image, -1, gaussian_kernel)

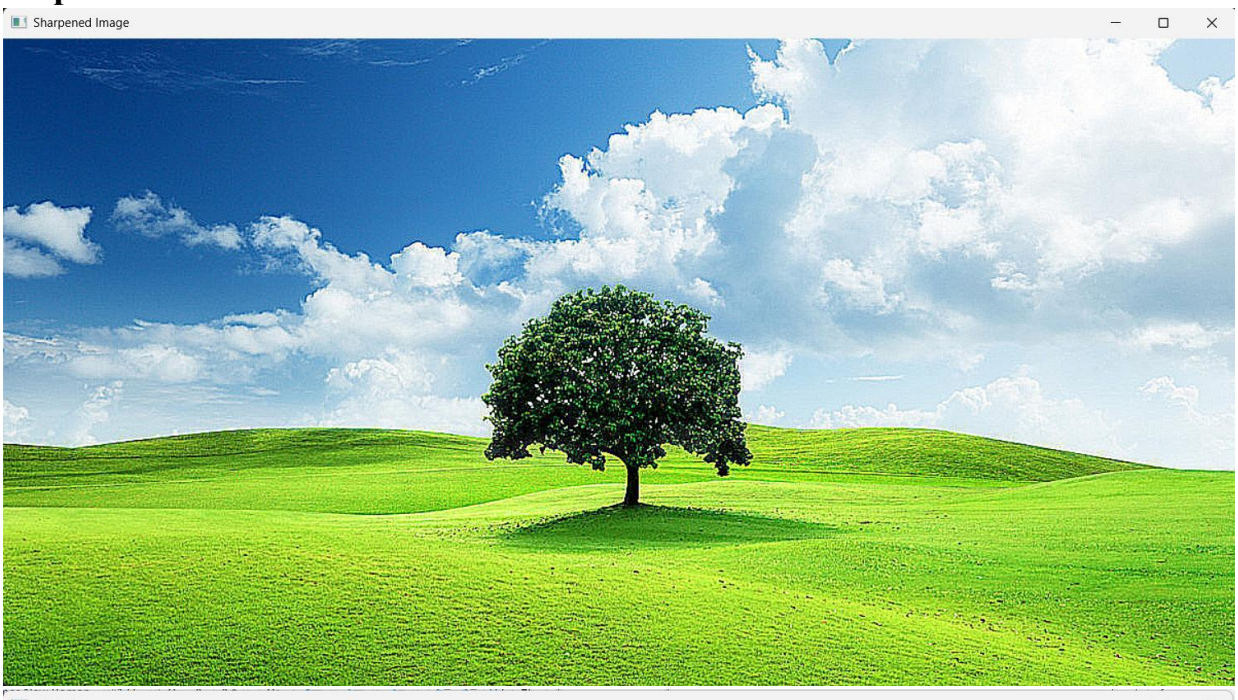
# Define a sharpening kernel
sharpening_kernel = np.array([[-1, -1, -1],
                              [-1, 9, -1],
                              [-1, -1, -1]])

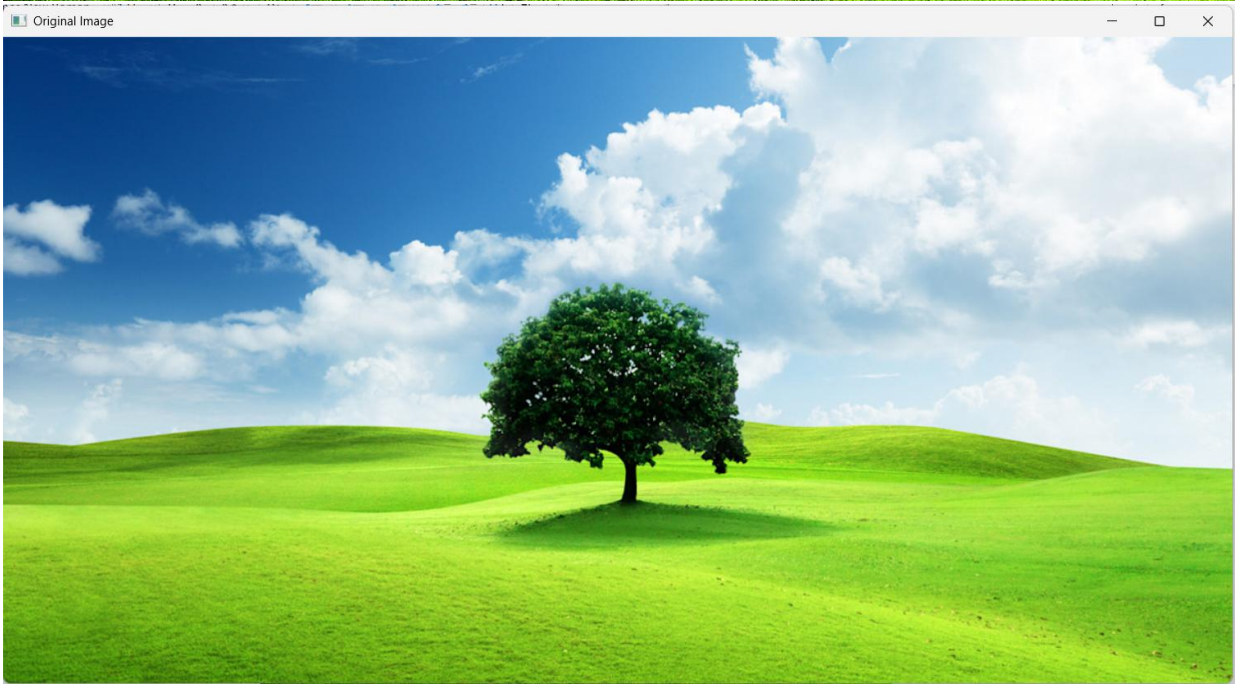
# Apply sharpening
sharpened_image = cv2.filter2D(image, -1, sharpening_kernel)

# Display the original image, smoothed, and sharpened images
cv2.imshow('Original Image', image)
cv2.imshow('Smoothed Image', smoothed_image)
cv2.imshow('Sharpened Image', sharpened_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology**	
Subject: DSIP	**Aim: Image processing_3**	
Experiment:-9	**Date:22-08-2026**	**Enrollment No:92400133063**

output:-





Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology**	
Subject: DSIP	**Aim: Image processing_3**	
Experiment:-9	**Date:22-08-2026**	**Enrollment No:92400133063**



Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology**	
Subject: DSIP	**Aim: Image processing_3**	
Experiment:-9	**Date:22-08-2026**	**Enrollment No:92400133063**

Aim:- write a python program to simulate smoothening using Averaging linear filters

Code:-

```
import cv2
import numpy as np

# Load the image
image = cv2.imread('free-nature-images.jpg')

# Define the size of the Averaging filter kernel
kernel_size = (5, 5)  # You can adjust the size based on the desired smoothing level

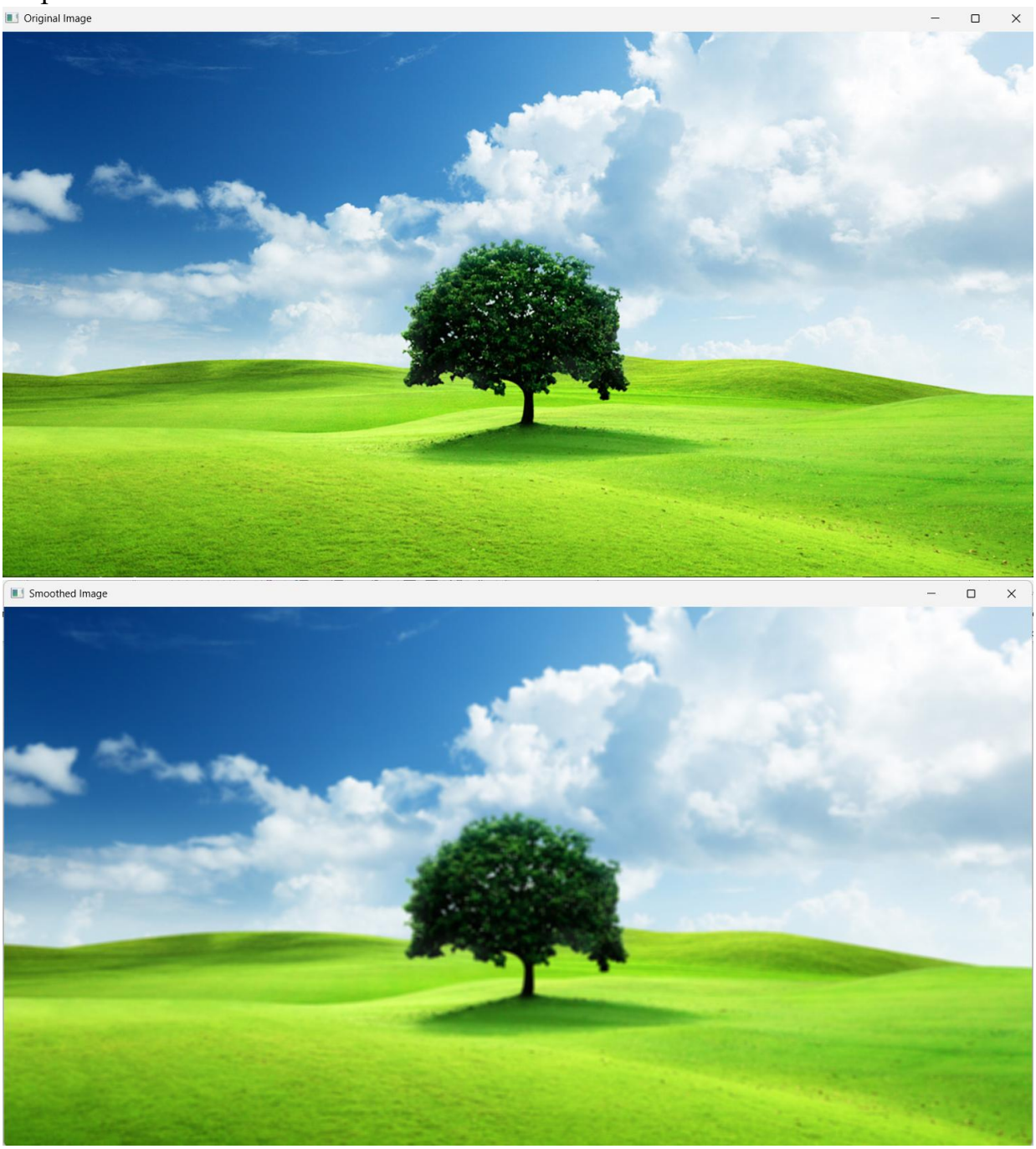
# Create the Averaging filter kernel
kernel = np.ones(kernel_size, dtype=np.float32) / (kernel_size[0] * kernel_size[1])

# Apply the Averaging filter for smoothing
smoothed_image = cv2.filter2D(image, -1, kernel)

# Display the original and smoothed images
cv2.imshow('Original Image', image)
cv2.imshow('Smoothed Image', smoothed_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology**	
Subject: DSIP	**Aim: Image processing_3**	
Experiment:-9	**Date:22-08-2026**	**Enrollment No:92400133063**

output:-



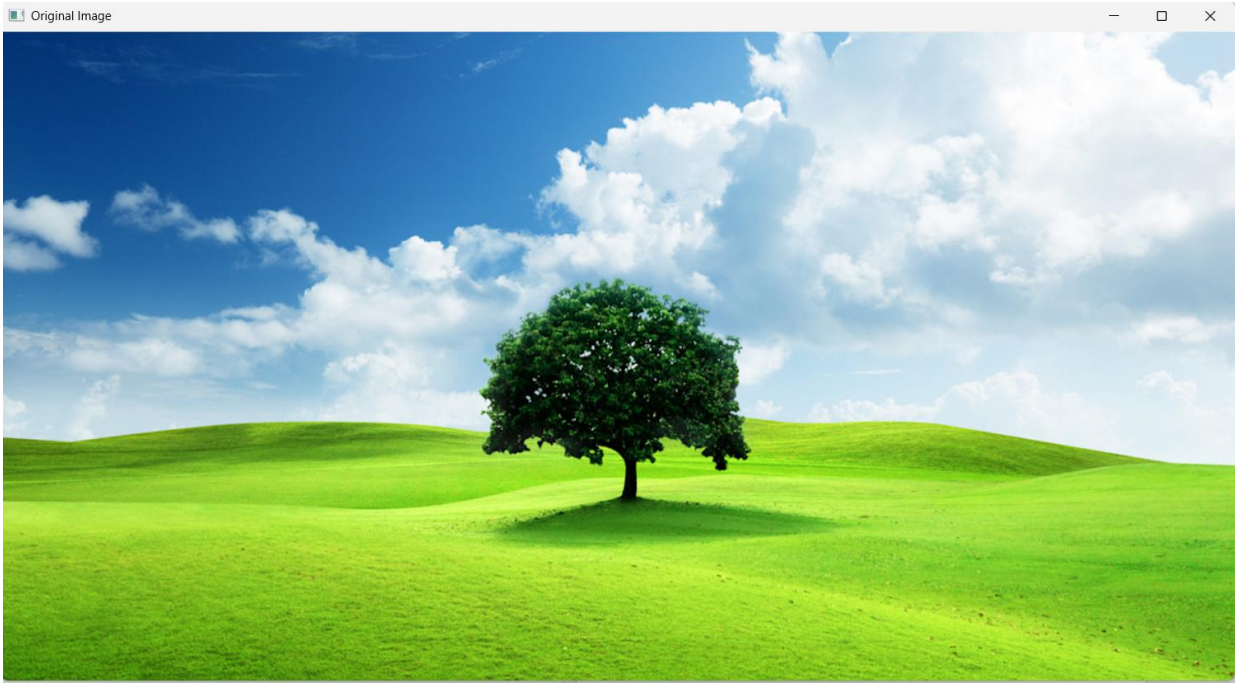
Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology**	
Subject: DSIP	**Aim: Image processing_3**	
Experiment:-9	**Date:22-08-2026**	**Enrollment No:92400133063**

Aim:- write a python program to simulate smoothening using median filters

Code:-

```
import cv2
import numpy as np
# Load the image
image = cv2.imread('free-nature-images.jpg')
# Define the size of the median filter kernel (should be an odd number)
kernel_size = 5  # You can adjust the size based on the desired smoothing level
# Apply the Median filter for smoothing
smoothed_image = cv2.medianBlur(image, kernel_size)
# Display the original and smoothed images
cv2.imshow('Original Image', image)
cv2.imshow('Smoothed Image', smoothed_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

output:-



Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology**	
Subject: DSIP	**Aim: Image processing_3**	
Experiment:-9	**Date:22-08-2026**	**Enrollment No:92400133063**



Aim:- write a python program to simulate sharpening using spatial high pass filters

Code:-

```
import cv2
import numpy as np

# Load the image
image = cv2.imread('free-nature-images.jpg')

# Apply Gaussian smoothing to reduce noise (optional but recommended)
blurred_image = cv2.GaussianBlur(image, (5, 5), 0)

# Create a Laplacian kernel for sharpening
laplacian_kernel = np.array([[0, -1, 0],
                             [-1, 5, -1],
                             [0, -1, 0]], dtype=np.float32)

# Apply the Laplacian filter for sharpening
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology**	
Subject: DSIP	**Aim: Image processing_3**	
Experiment:-9	**Date:22-08-2026**	**Enrollment No:92400133063**

```
sharpened_image = cv2.filter2D(blurred_image, -1, laplacian_kernel)

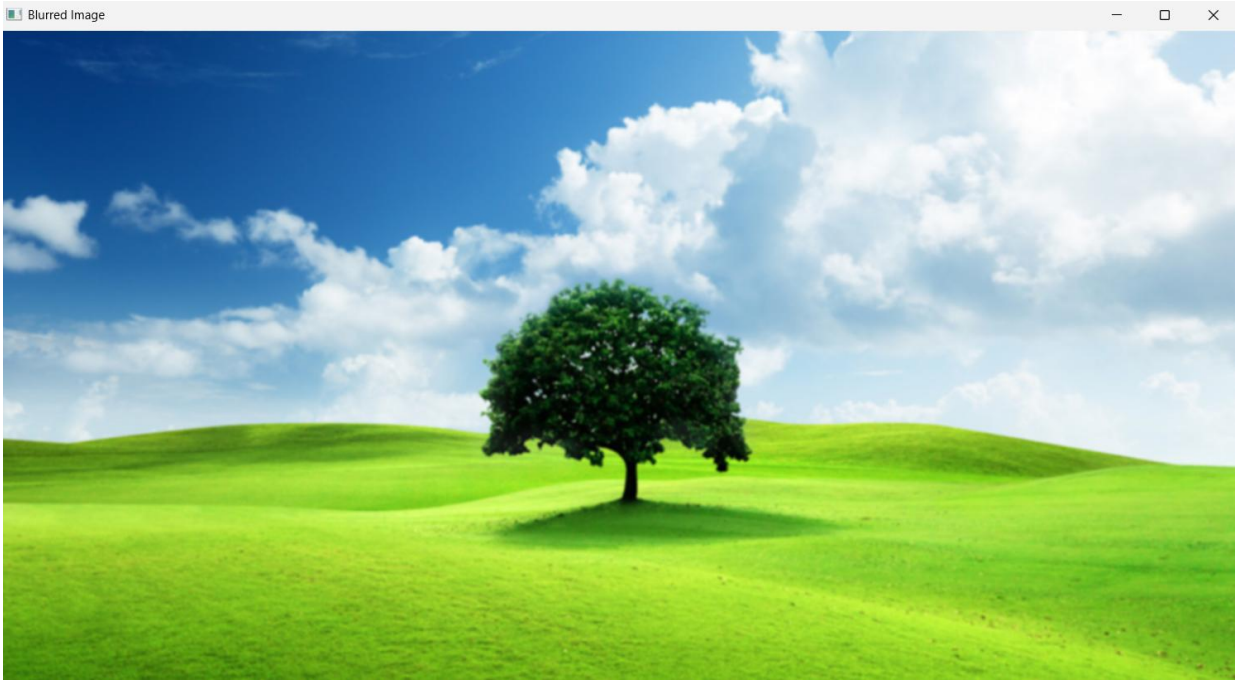
# Display the original image, blurred image, and sharpened image
cv2.imshow('Original Image', image)
cv2.imshow('Blurred Image', blurred_image)
cv2.imshow('Sharpened Image', sharpened_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

output:-



Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology**	
Subject: DSIP	**Aim: Image processing_3**	
Experiment:-9	**Date:22-08-2026**	**Enrollment No:92400133063**





Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology**	
Subject: DSIP	**Aim: Image processing_3**	
Experiment:-9	**Date:22-08-2026**	**Enrollment No:92400133063**

Conclusion:-

This experiment successfully demonstrates two fundamental image processing techniques: histogram equalization and histogram matching. Histogram equalization effectively enhances the contrast of a single image by redistributing pixel intensities, resulting in a more uniformly distributed histogram and a visually improved output. Furthermore, histogram matching successfully transfers the tonal characteristics of a reference image to a source image, allowing for stylization or standardization of image appearances. Both techniques highlight the power of manipulating pixel intensity distributions to achieve specific visual effects in image processing applications.